

# From Statistics to Deep Learning: A Progressive Time Series Approach for AI-Driven Rice Irrigation Forecasting and Sustainable Water Management

**Hammad Farid**

Assistant Professor, Department of Information and Communication Technology (ICT),  
Monark University, Vahelal, Ahmedabad, India

## **Abstract**

The world is already facing a shortage of fresh water, and extreme climate changes have worsened the situation. This has intensified the need for sustainable water management, especially for water-intensive crops like rice, which alone accounts for nearly 40% of the total water used for irrigation. Traditionally, farmers heavily depend on static calendars or their past experience for watering crops. However, these methods fail to adapt to continuously changing weather, which results in both water wastage and compromised crop productivity. To solve this, this chapter introduces an experimental and AI-driven irrigation framework. This framework uses historical weather and environmental data, covering many years of planting and harvesting, to forecast rice irrigation water requirements.

To find the best way to predict soil moisture, we tested our proposed framework with seven different models, moving from simple math to advanced AI. We started with basic statistical tools like AR, MA, and ARMA to establish a baseline in order to see how well simple math can predict the soil moisture. Then, we tested ARIMA and SARIMA to see if they could handle seasonal farming trends better. Finally, we applied deep learning networks Gated Recurrent Unit (GRU) and Long Short-Term Memory (LSTM) since they are designed to understand highly complex, long-term weather patterns.

We evaluated the models using 24 years (2000–2023) of real-world weather and soil data, which consist of temperature, rainfall, relative humidity, solar radiation, and soil moisture. To see which model performed best, we evaluated them using RMSE, MAE,  $R^2$ , and MAPE scores. We found that as the models became more advanced and complex, their accuracy improved. The GRU model was the clear winner, achieving an  $R^2$  of 0.9789 and beating the LSTM. Ultimately, this proves that deep learning can serve as a highly reliable tool for farmers. By using AI to schedule irrigation, we can save water and support global sustainability goals.

**Keywords:** Smart Irrigation, Time Series Forecasting, GRU, LSTM, SARIMA, Sustainable Agriculture, Deep Learning, Rice Cultivation, Soil Moisture Prediction, Water Management.

## **1 Introduction**

### **1.1 Background**

By the year 2050, the global population will reach 9.7 billion [1]. To feed everyone, we must produce more food, especially staple crops like rice, which feeds over 3.5 billion people [2].

However, growing rice requires a massive amount of resources. Agriculture currently consumes about 70% of the world’s freshwater [3], and rice cultivation alone takes up nearly 40% of all irrigation water [4]. Moreover, to worsen the situation, climate change makes seasonal rainfall highly unpredictable. Traditional, unmonitored irrigation methods waste too much water, creating an urgent need for systemic reform.

We see this crisis clearly in India. India is the second-largest rice producer in the world. Within the country, the state of West Bengal produces the most rice, making its local water management critical for national food security. Unfortunately, most of the farmers there still use conventional methods. Various studies show that farmers in the Bengal region lose between 30% and 40% of applied water simply by over-irrigating their fields [5]. This massive wastage of irrigation water is draining underground water reserves and threatening rural ecosystems as well. Because traditional farmers usually decide when to water based on personal experience rather than actual data, we urgently need smart, data-driven irrigation systems to fix this. These systems must analyze historical weather patterns to give farmers practical, accurate, and forward-looking irrigation schedules.

Recent advancements in artificial intelligence (AI) and machine learning provide a practical solution for precision agriculture. Today, anyone can easily access detailed environmental data. Platforms like NASA POWER offer comprehensive, free historical records on weather metrics and soil conditions. We can feed these time-series datasets into modern computational models to track complex atmospheric patterns and figure out exactly how the soil and weather change from day to day. Deep learning architectures are especially useful here. They naturally pick up on hidden, non-linear trends that traditional statistical methods usually miss. Ultimately, integrating these AI models allows us to accurately predict future irrigation needs. This data-driven approach optimizes water conservation and directly supports global goals for food and water security.

## 1.2 Motivation

Despite growing interest in precision agriculture, existing irrigation studies largely rely on simplified statistical models [6,7] that fail to capture highly dynamic environmental shifts. Furthermore, most prior literature prioritizes long-term crop yield prediction rather than the immediate, daily forecasting of localized irrigation requirements. Crucially, systematic comparisons combining classical statistical frameworks with advanced deep learning remain scarce. In particular, a progressive evaluation tracking performance incrementally across an evolutionary spectrum of models — moving from AR, MA, ARMA, ARIMA, and SARIMA to GRU and LSTM — remains entirely unexamined. This absence of an incremental benchmarking framework for daily soil moisture forecasting represents a distinct, unexplored research gap.

This chapter addresses this gap directly by executing a rigorous, step-by-step comparative analysis of time-series frameworks. This progressive approach isolates the exact computational threshold where deep learning architectures begin to outperform traditional statistical baselines. To ensure empirical reliability, the study utilizes 24 years of historical agro-meteorological data (2000–2023) from the NASA POWER platform [10], focused specifically on the vital rice-farming landscape of West Bengal, India. By selecting daily soil wetness as the primary target variable — serving as a direct indicator of irrigation needs — the outcomes deliver actionable, practical decision support for regional farmers and water resource policymakers.

To achieve this, the investigation fulfills five clear research objectives:

- **Data Aggregation:** Collect and preprocess 24 years of localized agro-meteorological data from the NASA POWER platform to establish a reliable baseline.
- **Model Implementation:** Deploy seven progressive time-series models spanning from foundational autoregressive frameworks to advanced recurrent deep learning architectures, encompassing AR, MA, ARMA, ARIMA, SARIMA, GRU, and LSTM.

- **Performance Evaluation:** Rigorously compare these frameworks using standardized metrics, specifically Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), Coefficient of Determination ( $R^2$ ), and Mean Absolute Percentage Error (MAPE).
- **Optimization Identification:** Pinpoint the most accurate and reliable forecasting model for daily soil wetness forecasting under dynamic weather conditions.
- **Actionable Strategy:** Deliver scalable recommendations to promote AI-driven, sustainable agricultural water management practices.

## 2 Computational Algorithms in AI

### 2.1 Classical Statistical Models

#### 2.1.1 AutoRegressive (AR) Model

The AutoRegressive (AR) model serves as a foundational baseline in time-series forecasting by predicting future values based on a linear combination of its own historical observations [11]. Mathematically, an  $AR(p)$  model is defined as:

$$Y_t = c + \sum_{i=1}^p \phi_i Y_{t-i} + \varepsilon_t \quad (1)$$

where  $Y_t$  represents the target variable at time  $t$ ,  $c$  is a constant,  $\phi_i$  denotes the autoregressive coefficients,  $p$  signifies the lag order, and  $\varepsilon_t$  is white noise error. The optimal lag order  $p$  is determined by analyzing the Partial Autocorrelation Function (PACF) plot, where a significant cutoff indicates the limit of direct historical influence. In this study, an  $AR(30)$  configuration was selected and trained on 7,012 daily observations after verifying data stationarity using the Augmented Dickey-Fuller (ADF) test. However, a primary limitation of the AR model is its strict reliance on linear historical dependencies, rendering it incapable of capturing non-linear patterns or multi-scale seasonal fluctuations.

#### 2.1.2 Moving Average (MA) Model

The Moving Average (MA) model approaches time-series forecasting by modeling the current value of a variable as a linear combination of past forecast errors rather than past values [11]. The structural equation for an  $MA(q)$  process is expressed as:

$$Y_t = \mu + \varepsilon_t + \sum_{j=1}^q \theta_j \varepsilon_{t-j} \quad (2)$$

where  $\mu$  is the true mean of the series,  $\theta_j$  represents the moving average weights,  $q$  is the lag order of the forecast error, and  $\varepsilon_{t-j}$  represents the historical residual error terms. The selection of the error lag parameter  $q$  is traditionally guided by the Autocorrelation Function (ACF) plot to identify where the correlation drops significantly. For this investigation, the framework was implemented via an  $ARIMA(0,0,15)$  configuration to focus exclusively on the error-correcting behavior of the daily agricultural series. Despite its utility in smoothing out random high-frequency noise, the MA model possesses a short memory span and cannot capture persistent autoregressive trends independently.

#### 2.1.3 AutoRegressive Moving Average (ARMA) Model

The AutoRegressive Moving Average (ARMA) model provides a more comprehensive framework by combining the lag properties of the AR model with the error-filtering capabilities of the MA

model [12]. The mathematical formulation of an ARMA( $p, q$ ) model merges these components into a single linear equation:

$$Y_t = c + \sum_{i=1}^p \phi_i Y_{t-i} + \sum_{j=1}^q \theta_j \varepsilon_{t-j} + \varepsilon_t \quad (3)$$

Here, the model simultaneously leverages the autoregressive order  $p$  and the moving average order  $q$  to map temporal dependencies. In this study, a balanced ARMA(5,5) setup was deployed to simultaneously capture both immediate historical values and recent forecast anomalies within the agro-meteorological dataset. A major disadvantage of this model, however, is that it strictly assumes variance and mean stationarity over time, preventing it from adapting to raw, non-stationary environmental data or shifting seasonal baselines.

#### 2.1.4 AutoRegressive Integrated Moving Average (ARIMA) Model

The AutoRegressive Integrated Moving Average (ARIMA) model builds upon the ARMA architecture by incorporating an integrated (I) component designed to eliminate non-stationarity through data differencing [12]. The generalized mathematical formulation for an ARIMA( $p, d, q$ ) model is expressed as:

$$\Delta^d Y_t = c + \sum_{i=1}^p \phi_i \Delta^d Y_{t-i} + \sum_{j=1}^q \theta_j \varepsilon_{t-j} + \varepsilon_t \quad (4)$$

where  $\Delta^d$  represents the differencing operator applied  $d$  times to achieve a constant mean across the time series. To identify the most effective combination of parameters, an automated search algorithm (Auto-ARIMA) was executed to evaluate multiple permutations of  $p$ ,  $d$ , and  $q$  based on the minimization of the Akaike Information Criterion (AIC). While the integration layer successfully tracks and eliminates linear trend shifts over long-term timelines, the standard ARIMA framework remains structurally constrained because it cannot interpret cyclical, calendar-based seasonal patterns.

#### 2.1.5 Seasonal ARIMA (SARIMA) Model

The Seasonal AutoRegressive Integrated Moving Average (SARIMA) model is an advanced extension of ARIMA engineered specifically to capture periodic, cyclical variations alongside standard linear trends [13]. It expands the baseline notation into a seasonal framework denoted as SARIMA( $p, d, q$ )( $P, D, Q$ ) $_m$ , defined mathematically as:

$$\Phi_P(B^m) \phi_p(B) \Delta_m^D \Delta^d Y_t = \Theta_Q(B^m) \theta_q(B) \varepsilon_t \quad (5)$$

where  $P$ ,  $D$ ,  $Q$  represent the seasonal autoregressive, differencing, and moving average orders respectively, while  $m$  signifies the explicit seasonal length, and  $B$  is the backshift operator. For this agricultural application, a weekly seasonal period of  $m = 7$  was defined to track repeating short-term cycles within the daily environmental variables. This configuration allows the model to process systematic fluctuations; however, its deterministic seasonal structure struggles significantly when confronted with the highly volatile, irregular monsoon seasons characteristic of the West Bengal climate region. This behavioral constraint, observed empirically in the present study, ultimately motivated the transition toward non-linear deep learning architectures capable of adaptive temporal learning.

### 2.1.6 Summary of Statistical Model Configurations

To provide a consolidated overview of the mathematical baselines utilized within this comparative framework, the specific structural parameters and selection protocols for each classical estimator are summarized in Table 1.

Table 1: Statistical Model Configurations

| Model  | Order            | Selection Method |
|--------|------------------|------------------|
| AR     | $p = 30$         | PACF Analysis    |
| MA     | $q = 15$         | ACF Analysis     |
| ARMA   | $p = 5, q = 5$   | Manual Tuning    |
| ARIMA  | Auto-selected    | AIC Minimization |
| SARIMA | $m = 7, (2,0,2)$ | Grid Search      |

## 2.2 Deep Learning Algorithms

### 2.2.1 Gated Recurrent Unit (GRU)

Introduced by Cho et al. in 2014 as part of their work on sequence-to-sequence learning [8], the Gated Recurrent Unit (GRU) functions as a highly streamlined variant of the traditional Recurrent Neural Network (RNN). Researchers engineered this architecture specifically to resolve the vanishing gradient problem inherent in sequential data training [8]. Unlike its predecessors, GRU operates entirely without a separate, distinct cell state. Instead, it relies on a dual-gating mechanism — comprising reset and update gates — to efficiently manage temporal information flow and process sequential arrays.

The reset gate dictates the extent to which historical information should be discarded. Its activation is mathematically defined as:

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r) \quad (6)$$

Here,  $\sigma$  represents the sigmoid activation function, mapping the output strictly between 0 and 1. When the value of  $r_t$  approaches zero, the model actively forgets past data, whereas a value near one forces the network to retain historical inputs.

Conversely, the update gate determines the optimal balance between newly acquired environmental data and existing memory. The update vector is computed as:

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z) \quad (7)$$

This scalar filters the candidate hidden state, formulated as:

$$\tilde{h}_t = \tanh(W \cdot [r_t \odot h_{t-1}, x_t] + b) \quad (8)$$

The architecture then synthesizes the final hidden state for the current timestep using the equation:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (9)$$

The experimental GRU framework relies on two stacked recurrent layers. The initial layer contains 64 units configured to return full sequences, immediately followed by a 0.2 dropout mechanism to limit overfitting. A subsequent 32-unit GRU layer connects to a 16-neuron dense layer employing ReLU activation. The final output layer condenses predictions into a single continuous variable. Optimization relies on the Adam algorithm minimizing Mean Squared

Error (MSE). The network ingests a 30-day lookback sequence, utilizing an early stopping patience of 10 epochs. This structural efficiency culminates in a computationally lightweight model requiring only 22,817 trainable parameters.

### 2.2.2 Long Short-Term Memory (LSTM)

Originally proposed by Hochreiter and Schmidhuber in 1997 [9], the Long Short-Term Memory (LSTM) network was specifically designed to capture extensive long-range dependencies within complex sequential datasets. Subsequent research has validated LSTM as one of the most robust architectures for agricultural and environmental time series applications [14]. The primary structural divergence separating LSTM from GRU is its explicit maintenance of an internal cell state. This added dimensionality requires three independent regulatory gates — forget, input, and output — making the architecture considerably more expressive and ubiquitous in robust time series tasks.

The forget gate evaluates previous hidden states and current inputs to discard irrelevant cell state data. The mathematical trigger is defined as:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (10)$$

Outputting a 0 forces complete erasure of that specific memory block, while a 1 triggers total retention, a dynamic necessary for processing extended agricultural sequences.

The input gate identifies which new measurements warrant storage, quantified by:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (11)$$

Simultaneously, the network generates a candidate cell state vector:

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad (12)$$

The overarching long-term memory mechanism combines these elements to refresh the internal state:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (13)$$

Finally, the output gate dictates what fraction of the updated cell state visibly emerges as the hidden state. This control matrix is:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (14)$$

The final hidden state, defined as  $h_t = o_t \odot \tanh(C_t)$ , branches forward into the subsequent timestep while simultaneously feeding the dense prediction layers.

To ensure strict experimental parity, the LSTM implementation mirrors the structural depth of the GRU network. Two stacked layers process the same 30-day lookback window; the first contains 64 sequence-returning units, and the second contains 32 units, with identical 0.2 dropout phases separating them. A 16-neuron ReLU dense layer funnels the extracted temporal features into a single-node output. Trained via the Adam optimizer against MSE loss with a 10-epoch early stopping threshold, the inclusion of the separate cell state pushes the total trainable parameter count to 29,857.

### 2.2.3 Comparative Overview of Deep Learning Architectures

Table 2 outlines the mechanical and performance distinctions between the two deployed deep learning configurations.

The architectural simplicity of GRU, reflected in its reduced parameter count of 22,817 compared to LSTM’s 29,857, proved particularly advantageous for the univariate soil wetness

Table 2: Architectural Comparison: GRU versus LSTM

| Feature              | GRU               | LSTM                      |
|----------------------|-------------------|---------------------------|
| Gating Mechanisms    | 2 (Reset, Update) | 3 (Forget, Input, Output) |
| Separate Cell State  | No                | Yes                       |
| Trainable Parameters | 22,817            | 29,857                    |
| Training Speed       | Faster            | Slower                    |
| Lookback Window      | 30 days           | 30 days                   |
| Optimizer            | Adam              | Adam                      |
| Loss Function        | MSE               | MSE                       |
| Best $R^2$ (Study)   | 0.9789            | 0.9557                    |
| Best RMSE (Study)    | 0.0282            | 0.0408                    |

series examined in this study. This finding is consistent with recent literature [15] suggesting that GRU’s streamlined gating mechanism can prevent overfitting on moderately complex univariate sequences, a conclusion elaborated further in Section 4.

### 3 Applications and Experimental Methodology

#### 3.1 Dataset Acquisition and Profiling

The empirical foundation of this study relies on long-term agro-meteorological records extracted from the NASA Prediction of Worldwide Energy Resources (POWER) platform [10]. The geographical perimeter isolates the agricultural zones surrounding Kolkata, West Bengal (Latitude: 22.57°N, Longitude: 88.37°E), a critical region for high-yield rice cultivation. The extracted continuous timeline spans exactly 24 years, capturing daily sequential observations from January 1, 2000, to December 31, 2023. This extraction yielded a cumulative matrix of 8,766 daily records. This extended observational window deliberately encompasses multiple complete monsoon cycles, ensuring the sequential models encounter sufficient climatic variability during training.

The multivariate feature space comprises seven distinct climatological parameters: Maximum Temperature (°C), Minimum Temperature (°C), Relative Humidity (%), Precipitation (mm/day), Wind Speed (m/s), Solar Radiation (MJ/m<sup>2</sup>/day), and the primary predictive target, Surface Soil Wetness. During the initial data retrieval phase, absent instrumental readings designated by the system default code -999 were identified. These anomalies were systematically resolved using a forward-fill interpolation technique to preserve chronological continuity without distorting underlying temporal trends. A comprehensive descriptive statistical breakdown of these finalized variables is formally cataloged in Section 4.

#### 3.2 Data Preprocessing and Transformation Pipeline

Raw dataset ingestion required a structured preprocessing pipeline to ensure strict algorithmic compatibility. Initial parsing involved the truncation of 15 metadata header rows to isolate the core numerical matrix. Temporal indexing was reconstructed by fusing the independent ‘YEAR’ and Day-of-Year (‘DOY’) vectors into a standardized chronological datetime index. Subsequent diagnostic verification confirmed zero remaining null values across the matrix following the initial interpolation protocol.

To evaluate temporal stability, the Augmented Dickey-Fuller (ADF) statistical test was applied directly to the target Soil Wetness array. The diagnostic yielded a p-value of 0.0000, confirming strict data stationarity and instantly eliminating the requirement for any differential

transformations (i.e., parameter  $d = 0$  for integrated models). The validated timeline was partitioned chronologically utilizing an 80:20 ratio. The training corpus consumed the initial 7,012 sequences (spanning 2000–2019), sequentially allocating the residual 1,754 records (spanning 2019–2023) for unseen validation and testing.

For the advanced deep learning architectures specifically, the numerical bounds were compressed using a `MinMaxScaler` function to enforce a normalized  $[0, 1]$  feature range, a mandatory step for mitigating gradient vanishing risks inside recurrent nodes. Finally, temporal sequence generators restructured this normalized array into overlapping 30-day lookback windows, explicitly formulating the 3D tensor dimensions required for GRU and LSTM ingestion.

### 3.3 Experimental Environment and Software Architecture

All computational simulations and predictive modeling tasks were executed within the Google Colaboratory cloud environment [16], leveraging hardware-accelerated GPU processing instances. The underlying software stack operated on a Python 3.12 base interpreter. Classical statistical models were compiled via the `statsmodels` library, supplemented by `pmdarima` for automated AIC-based parameter optimization. Conversely, the recurrent neural network topologies were synthesized using the TensorFlow and Keras deep learning frameworks. Matrix operations and feature scaling relied exclusively on `numpy`, `pandas`, and `scikit-learn`, while performance visualizations were rendered via `matplotlib` and `seaborn`. Absolute experimental reproducibility was strictly enforced by globally anchoring all pseudorandom number generators to an initialization seed of 42.

### 3.4 Evaluation Metrics

To rigorously quantify algorithmic precision, the framework computes four standard statistical evaluation metrics measuring the deviation between predicted outputs ( $\hat{y}_i$ ) and actual target values ( $y_i$ ) over  $n$  observations.

The Root Mean Squared Error (RMSE) disproportionately penalizes severe prediction anomalies by squaring the residual terms prior to averaging, making it a critical metric for agricultural safety boundaries:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (15)$$

The Mean Absolute Error (MAE) provides a strictly linear, outlier-robust calculation of average prediction deviation:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (16)$$

The Coefficient of Determination ( $R^2$ ) calculates the total variance fraction successfully mapped by the predictive architecture, serving as the primary indicator of model fit:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (17)$$

Finally, the Mean Absolute Percentage Error (MAPE) normalizes the residual errors into a universally interpretable percentage ratio, indicating practical field reliability:

$$MAPE = \frac{100}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \quad (18)$$



### 3.5 Smart Irrigation Decision Framework

The practical translation of soil wetness forecasts into actionable irrigation decisions follows a threshold-based protocol. Predicted values below 0.40 signal critical moisture deficit, triggering immediate irrigation intervention. Values between 0.40 and 0.70 indicate moderate soil saturation, warranting continued monitoring without immediate action. Readings exceeding 0.70 confirm adequate moisture reserves, eliminating unnecessary water application. This tiered decision architecture, powered by the 30-day lookback window, enables farmers and automated IoT-based irrigation controllers to optimize daily water allocation with measurable precision.

## 4 Results and Discussion

The descriptive statistics cataloged in Table 3 provide a baseline understanding of the regional agro-meteorological dynamics. The target variable, Surface Soil Wetness, registers a mathematical mean of 0.67, indicating a general state of moderate soil saturation across the 24-year timeline. However, the atmospheric inputs dictating this moisture level exhibit extreme volatility; precipitation parameters fluctuate dramatically between absolute zero and maximum daily deluges of 171.86 mm, directly reflecting the intense, episodic nature of the South Asian monsoon cycle. Concurrently, thermal metrics display distinct seasonal demarcations, ranging from 16.85°C to 46.81°C. This pronounced variability immediately invalidates static agricultural assumptions and necessitates the deployment of sophisticated time series architectures capable of tracking non-linear climatic shifts.

Table 3: Descriptive Statistics of the Rice Irrigation Dataset (2000–2023)

| Variable         | Mean  | Std Dev | Min   | Max    |
|------------------|-------|---------|-------|--------|
| Max Temp (°C)    | 31.28 | 4.83    | 16.85 | 46.81  |
| Min Temp (°C)    | 21.25 | 6.23    | 3.43  | 30.35  |
| Humidity (%)     | 74.00 | 14.16   | 22.00 | 96.26  |
| Rainfall (mm)    | 4.32  | 9.31    | 0.00  | 171.86 |
| Wind Speed (m/s) | 2.19  | 1.02    | 0.36  | 9.14   |
| Solar Radiation  | 16.39 | 4.53    | 1.63  | 27.51  |
| Soil Wetness     | 0.67  | 0.20    | 0.17  | 1.00   |

Figure 1 visualizes the temporal distributions and inter-variable correlations across the 24-year study period, confirming that the underlying environmental drivers are highly erratic and exhibit pronounced seasonal patterns driven primarily by the South Asian monsoon cycle.

Before initiating predictive modeling, temporal stability was verified through the Augmented Dickey-Fuller (ADF) diagnostic test. The resulting computation returned a p-value of 0.0000, definitively confirming strict stationarity within the daily soil moisture array. This outcome mathematically eliminates the requirement for differential transformations, establishing an optimal foundation for the direct application of classical baseline algorithms like AutoRegressive (AR) and Moving Average (MA) configurations without risking unit root interference.

The performance metrics for all seven predictive frameworks are quantitatively synthesized in Table 4. Initial evaluations of the foundational statistical estimators exposed severe structural limitations. The AR and MA models produced Root Mean Squared Errors (RMSE) of 0.1948 and 0.1950, respectively. More significantly, their Coefficient of Determination ( $R^2$ ) values registered below zero (-0.0094 and -0.0114). Within predictive statistics, a negative  $R^2$  indicates that the deployed algorithm performs worse than a horizontal line representing the simple historical mean. While advancing to ARMA and ARIMA reduced the RMSE fractionally to 0.1942 and 0.1939, the corresponding  $R^2$  scores failed to breach zero. Across this entire classical

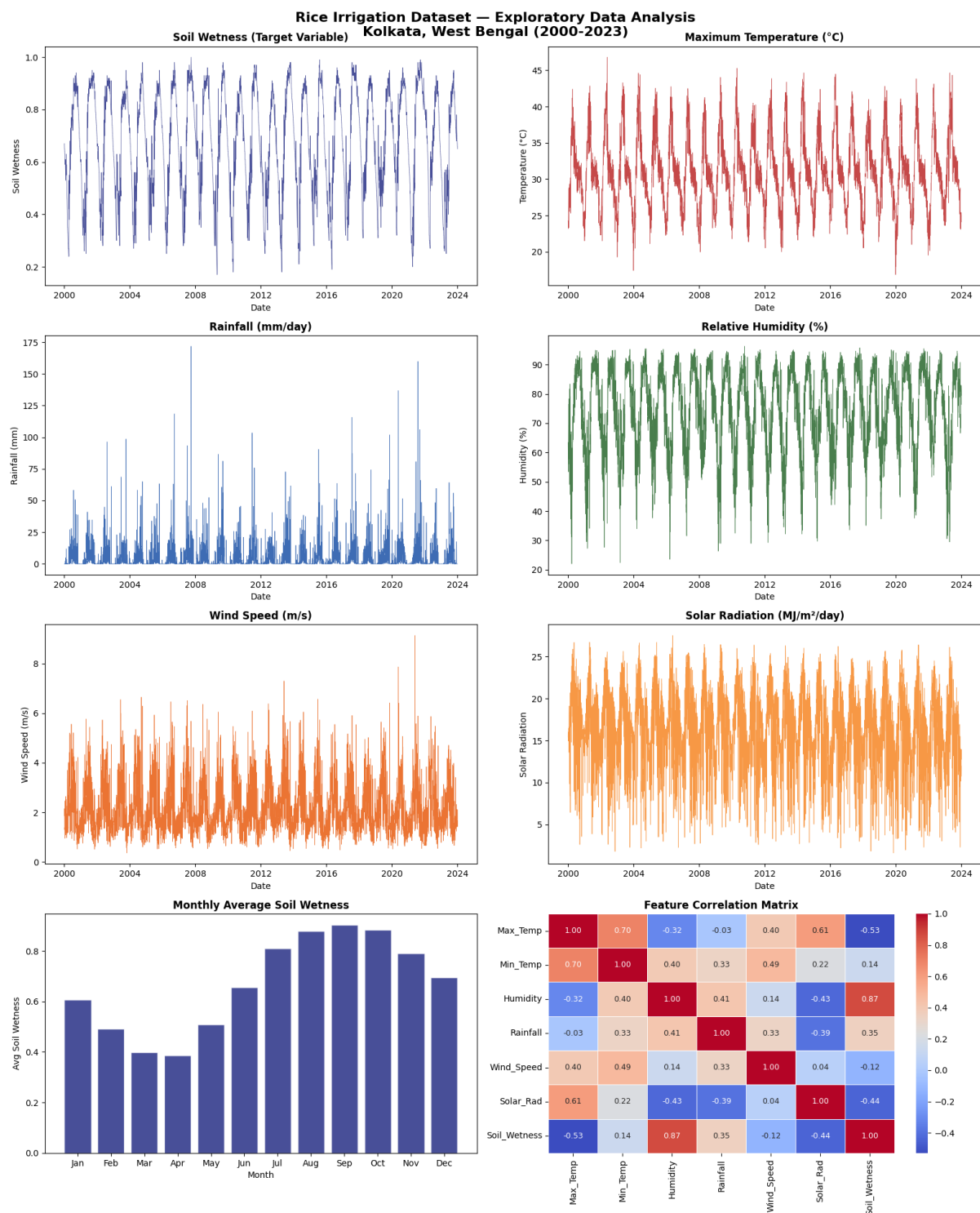


Figure 1: Exploratory Data Analysis: Temporal trends and feature correlation matrix of the agro-meteorological dataset (2000–2023), Kolkata, West Bengal, India.

tier, the Mean Absolute Percentage Error (MAPE) stagnated near the 30% threshold. These metrics confirm that linear, auto-correlated regressions cannot map the complex dependencies governing daily agricultural water retention.

The most dramatic failure within the classical hierarchy manifested during the Seasonal ARIMA (SARIMA) deployment. Despite its specific design targeting periodic fluctuations, SARIMA returned the lowest accuracy of the entire study, yielding an RMSE of 0.3963 and an exceptionally poor  $R^2$  of -3.1781. This quantitative collapse traces directly to the mathematical rigidity of the algorithm. SARIMA enforces a fixed seasonal period (configured here as  $m = 7$ ); however, the West Bengal climate region is dictated by erratic monsoon seasons where annual precipitation onset and withdrawal vary unpredictably by weeks. Forcing an irregular, non-deterministic environmental cycle into a rigid mathematical periodic structure fundamentally misaligns the forecast trajectory. Documenting this specific mechanistic failure provides vital literature context, proving that traditional seasonal estimators are incompatible with highly dynamic, monsoon-driven agricultural modeling.

Table 4: Final Model Comparison Metrics for Soil Wetness Prediction

| Model      | RMSE          | MAE           | $R^2$         | MAPE (%)    |
|------------|---------------|---------------|---------------|-------------|
| AR         | 0.1948        | 0.1691        | -0.0094       | 30.69       |
| MA         | 0.1950        | 0.1694        | -0.0114       | 30.66       |
| ARMA       | 0.1942        | 0.1671        | -0.0030       | 29.93       |
| ARIMA      | 0.1939        | 0.1668        | 0.0000        | 29.91       |
| SARIMA     | 0.3963        | 0.3441        | -3.1781       | 46.36       |
| <b>GRU</b> | <b>0.0282</b> | <b>0.0167</b> | <b>0.9789</b> | <b>3.02</b> |
| LSTM       | 0.0408        | 0.0297        | 0.9557        | 4.69        |

Transitioning to deep learning architectures precipitated an immediate, exponential leap in forecasting precision. The neural networks successfully decoded the non-linear climatic interactions that defeated the classical algorithms. The LSTM architecture reduced the RMSE to 0.0408, while the GRU model pushed the error down to an unprecedented 0.0282. Both networks achieved  $R^2$  scores exceeding 0.95, successfully mapping over 95% of the dataset’s total variance. Practically, this technological shift compressed the daily prediction error from a 30% baseline down to an operational 3.02% to 4.69% margin. This near tenfold improvement isolates the 30-day sequential lookback window as a decisive architectural advantage, allowing the recurrent nodes to internalize complex weather patterns accumulating over the preceding month to forecast next-day soil saturation accurately.

The actual versus predicted soil wetness values generated by the GRU model across the test period (2019–2023) are illustrated in Figure 2. The close alignment between forecast and observed trajectories visually confirms the model’s capacity to track both gradual seasonal transitions and abrupt monsoon-driven moisture spikes.

For comparative reference, Figure 3 presents the corresponding LSTM prediction trajectory. While the LSTM also demonstrates strong tracking ability, marginal deviations during peak monsoon periods reflect the minor overfitting penalty introduced by its larger parameter space.

The most compelling empirical finding of the comparative analysis contradicts prevalent assumptions regarding recurrent network hierarchies. While LSTM is widely presumed superior due to its intricate internal memory cells, the experimental results demonstrate that the GRU ( $R^2=0.9789$ ) definitively outperformed the LSTM ( $R^2=0.9557$ ). This inversion of expected hierarchy stems directly from architectural parsimony. The GRU relies on a streamlined dual-gate mechanism requiring only 22,817 trainable parameters, whereas the LSTM demands 29,857 parameters to manage its independent cell state. For a continuous univariate time series, the excessive dimensionality of the LSTM introduces minor overfitting friction, degrading

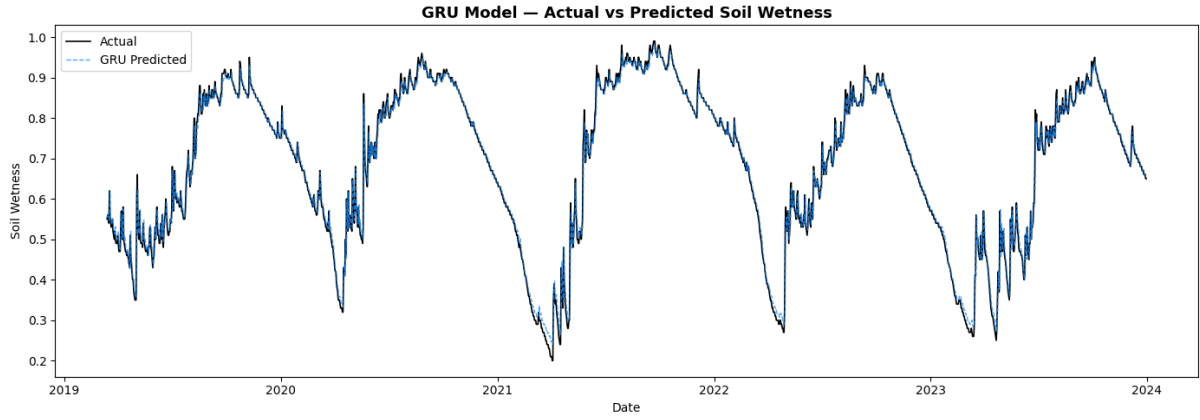


Figure 2: GRU Model: Actual versus Predicted Soil Wetness values across the test period (March 2019 – December 2023). The predicted trajectory closely tracks observed moisture dynamics including monsoon onset and withdrawal cycles.

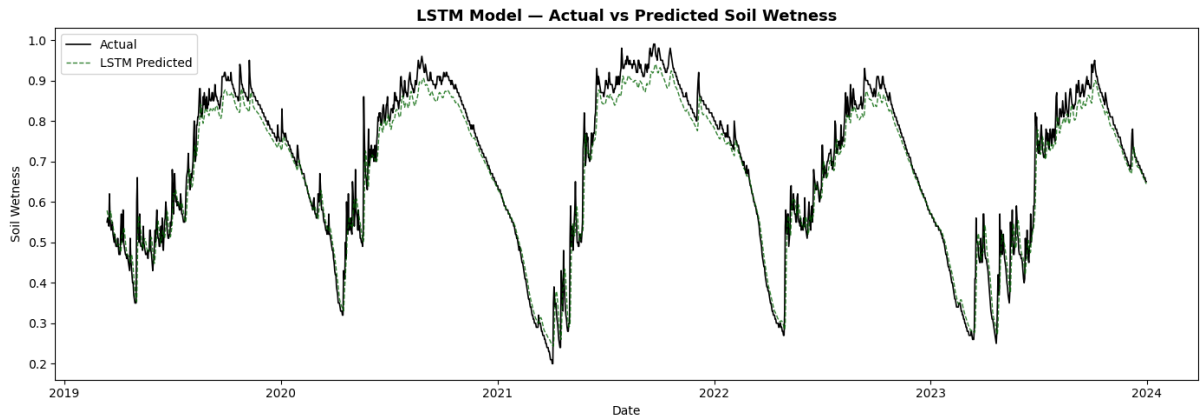


Figure 3: LSTM Model: Actual versus Predicted Soil Wetness values across the test period (March 2019 – December 2023), demonstrating strong but marginally lower predictive precision compared to the GRU architecture.

out-of-sample precision. The GRU maintains sufficient temporal memory capacity without the overfitting penalty, validating the principle of parsimony. This outcome directly aligns with the comparative benchmarks published by Greff et al. [15], confirming that isolated target variables favor simplified gating mechanics. Diagnostic training logs reveal both architectures triggered early stopping protocols at exactly epoch 18, indicating identical convergence speeds despite the final divergence in predictive accuracy.

Figure 4 presents the training and validation loss convergence curves for both GRU and LSTM architectures, confirming stable learning trajectories without evidence of significant overfitting across the training corpus.

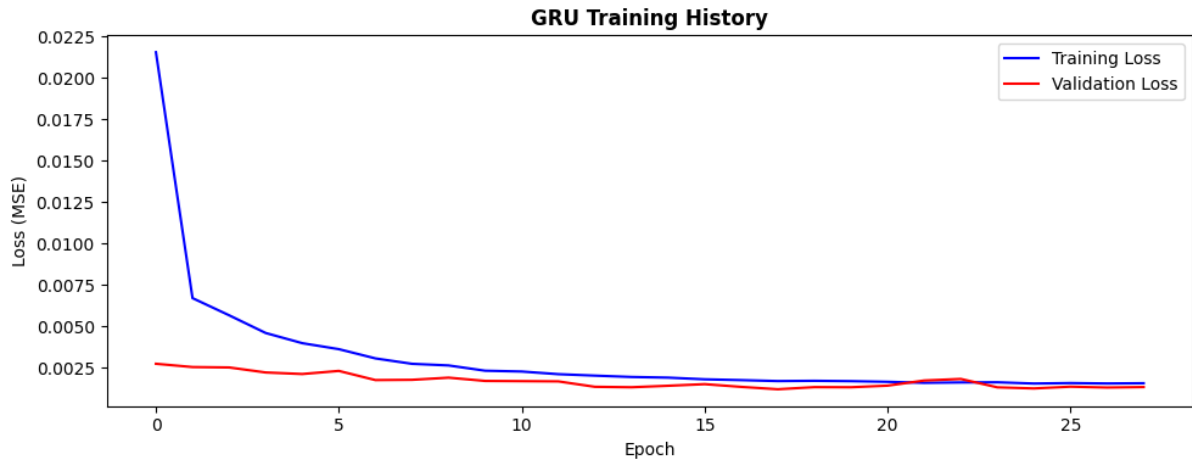


Figure 4: GRU Training History: Training and validation loss (MSE) convergence across epochs, with early stopping triggered at epoch 18. The stable gap between training and validation curves confirms controlled generalization.

Figure 5 presents a comprehensive visual comparison of RMSE and  $R^2$  metrics across all seven evaluated models, clearly illustrating the dramatic performance discontinuity between classical statistical estimators and deep learning architectures.

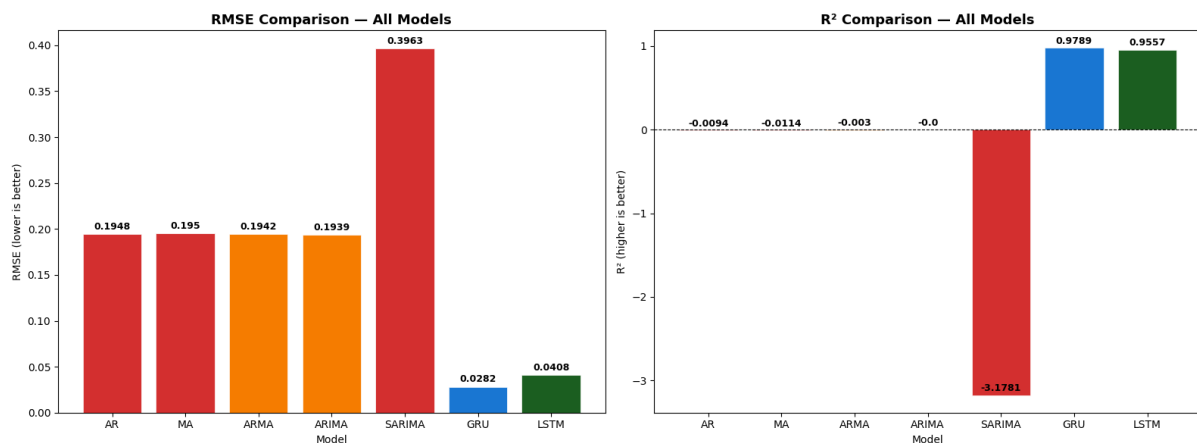


Figure 5: Performance Comparison: RMSE (left) and  $R^2$  (right) metrics across all seven time series models, illustrating the dramatic accuracy improvement achieved by transitioning from classical statistical estimators to deep learning architectures for daily soil wetness forecasting.

From a practical deployment perspective, the GRU model's predictions can be directly integrated into a threshold-based irrigation decision protocol. Forecast values below 0.40 signal critical moisture deficit requiring immediate irrigation, values between 0.40 and 0.70 indicate

adequate monitoring conditions, while predictions exceeding 0.70 confirm sufficient soil saturation, eliminating unnecessary water application entirely.

Ultimately, the quantitative gap between the evaluated paradigms dictates a mandatory shift in agricultural management strategies. The systemic inability of linear and fixed-seasonal estimators to process climatic volatility renders them obsolete for daily precision farming applications. The breakthrough accuracy of the Gated Recurrent Unit establishes a viable, highly reliable computational engine capable of translating erratic environmental inputs into stable, actionable irrigation schedules.

Beyond computational accuracy, these findings carry significant implications for global sustainability frameworks. The ability to forecast daily irrigation requirements with 97.89% accuracy directly supports United Nations Sustainable Development Goal 2 (Zero Hunger) by stabilizing rice yields against climate volatility, and SDG 6 (Clean Water and Sanitation) by substantially reducing unnecessary freshwater consumption. Deploying GRU-based forecasting infrastructure across West Bengal’s rice cultivation zones could potentially recover a significant fraction of the estimated 30%–40% water currently lost through over-irrigation practices [5], translating computational precision directly into measurable environmental and humanitarian impact.

## 5 Conclusion

This chapter systematically investigated a progressive time series forecasting framework engineered to optimize rice irrigation scheduling. The experimental architecture evaluated a hierarchical continuum of seven predictive algorithms, advancing sequentially from classical baselines (AR, MA, ARMA, ARIMA, SARIMA) to complex deep learning topologies (GRU, LSTM). Relying on 24 years of continuous, daily agro-meteorological observations extracted from the NASA POWER platform (2000–2023), the research specifically targeted the critical agricultural zones of West Bengal, India. Surface soil wetness served as the primary predictive variable, functioning as a direct proxy for field-level water retention and daily irrigation demand.

Empirical evaluations isolated distinct computational thresholds across the modeled paradigms. All linear statistical estimators structurally failed to capture the non-linear climatic variations, producing negative  $R^2$  values and baseline error rates (MAPE) approximating 30%. The SARIMA algorithm registered the most severe quantitative collapse, demonstrating that deterministic seasonal configurations cannot adapt to highly irregular monsoon onset cycles. Conversely, transitioning to recurrent neural networks precipitated a near tenfold reduction in predictive error. Both deep learning architectures successfully exceeded an  $R^2$  threshold of 0.95. Contradicting prevalent architectural assumptions, the streamlined GRU ( $R^2=0.9789$ ) measurably outperformed the LSTM ( $R^2=0.9557$ ), proving that reduced gating complexity mitigates overfitting risks within continuous univariate environmental series.

Translating these quantitative achievements into operational agriculture establishes the GRU architecture as a highly viable computational engine for real-world deployment. Utilizing the validated 30-day lookback window, farming cooperatives can proactively schedule water distribution, shifting away from reactive, experience-based methodologies. Integrating this predictive framework directly into automated IoT irrigation controllers introduces the immediate potential to recover the estimated 30% to 40% of freshwater currently lost to manual over-irrigation [5]. Consequently, state agricultural departments possess empirical justification to subsidize AI-driven decision-support systems, creating scalable policy frameworks applicable to equivalent rice-producing regions globally.

Despite the documented predictive accuracy, the current framework maintains specific operational boundaries. The experimental scope remains geographically isolated to the climatological parameters of West Bengal, limiting immediate pan-continental generalization. Algorithmically, the neural networks processed a strictly univariate target, excluding secondary botanical vari-

ables such as real-time root depth or explicit crop phenology stages. Furthermore, the simulations operated exclusively on historical satellite aggregates, lacking the localized noise and calibration challenges inherent in real-time, on-site IoT soil sensor hardware. True operational validation ultimately requires extensive, multi-season physical field testing.

Subsequent research trajectories must dismantle these constraints by expanding the geographic training matrix across diverse Indian agricultural states. Methodological evolution requires the ingestion of localized IoT hardware feeds and explicit crop growth parameters to formulate a fully multivariate feature space. Computational exploration will evaluate advanced CNN-LSTM hybrid topologies to simultaneously process spatial meteorological maps alongside temporal vectors. To ensure technological adoption at the grassroots level, these predictive engines must be embedded within accessible mobile application interfaces. Parallel expansion phases will execute transfer learning protocols, testing the GRU architecture’s adaptability to alternative water-intensive crops including maize and wheat, while integrating Explainable AI (XAI) layers to establish algorithmic trust among end-users.

Ultimately, precision forecasting architectures represent a mandatory technological pivot for modern agricultural sustainability. Deploying optimized recurrent networks to govern resource allocation directly advances United Nations Sustainable Development Goals 2 and 6 by simultaneously protecting critical food yields and preserving finite freshwater reservoirs. The transition from statistical estimation to deep learning does not merely enhance mathematical accuracy; it equips vulnerable agricultural ecosystems with the computational resilience necessary to withstand accelerating global climate volatility.

## References

- [1] United Nations Department of Economic and Social Affairs, “World Population Prospects 2022,” United Nations, New York, USA, Tech. Rep., 2022.
- [2] Food and Agriculture Organization of the United Nations, “The State of Food and Agriculture 2020,” FAO, Rome, Italy, Tech. Rep., 2020.
- [3] UNESCO World Water Assessment Programme, “The United Nations World Water Development Report 2021: Valuing Water,” UNESCO, Paris, France, 2021.
- [4] International Rice Research Institute, “Water-Saving Technologies for Rice Production,” IRRI, Los Baños, Philippines, Tech. Rep., 2020.
- [5] S. Kumar and R. Sharma, “Assessment of irrigation water use efficiency in rice cultivation in Eastern India,” *Agricultural Water Management*, vol. 245, pp. 106–118, 2021.
- [6] A. Mishra and V. Singh, “A review of drought concepts and statistical models for agricultural applications,” *Journal of Hydrology*, vol. 391, no. 1–2, pp. 202–216, 2020.
- [7] P. Sharma, N. Gupta, and R. Verma, “Classical time series models for agricultural forecasting: A review,” *Computers and Electronics in Agriculture*, vol. 178, pp. 105–118, 2020.
- [8] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using RNN encoder-decoder for statistical machine translation,” in *Proc. 2014 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, Doha, Qatar, 2014, pp. 1724–1734.
- [9] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.

- [10] NASA Langley Research Center, “POWER Project: Prediction of Worldwide Energy Resources,” National Aeronautics and Space Administration, Hampton, VA, USA. [Online]. Available: <https://power.larc.nasa.gov>
- [11] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Hoboken, NJ, USA: Wiley, 2015.
- [12] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: <https://otexts.com/fpp3>
- [13] P. J. Brockwell and R. A. Davis, *Introduction to Time Series and Forecasting*, 3rd ed. New York, NY, USA: Springer, 2016.
- [14] W. Zhao, J. Gao, T. Ji, X. Wan, Y. Wu, and L. Ye, “Deep temporal convolutional networks for short-term traffic flow forecasting,” *IEEE Access*, vol. 7, pp. 114496–114507, 2019.
- [15] K. Greff, R. K. Srivastava, J. Koutnik, B. R. Steunebrink, and J. Schmidhuber, “LSTM: A search space odyssey,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 28, no. 10, pp. 2222–2232, Oct. 2017.
- [16] Google Research, “Colaboratory: Google’s Jupyter notebook environment,” Google LLC, Mountain View, CA, USA, 2023. [Online]. Available: <https://colab.research.google.com>